LOGIN;

```
import React, { useState } from "react";
import { Link } from "react-router-dom";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { LogIn, Mail, Lock, Loader2 } from "lucide-react";
import AuthLayout from "@/components/AuthLayout";
import GoogleIcon from "@/components/GoogleIcon";
import { safeReturnTo } from "@/lib/authReturnTo";

export default function Login() {
 const [email, setEmail] = useState("");
 const [password, setPassword] = useState("");
 const [error, setError] = useState("");
 const [loading, setLoading] = useState(false);
 // Post-login destination (e.g. the MCP OAuth consent page sends users here
 // with returnTo so the grant flow can resume). Same-origin paths only.
 const returnTo = safeReturnTo();

 const handleSubmit = async (e) => {
  e.preventDefault();
  setError("");
  setLoading(true);
  try {
   await base44.auth.loginViaEmailPassword(email, password);
   window.location.href = returnTo;
  } catch (err) {
   setError(err.message || "Invalid email or password");
  } finally {
   setLoading(false);
  }
 };

 const handleGoogle = () => {
  base44.auth.loginWithProvider("google", returnTo);
 };

 return (
  <AuthLayout
   icon={LogIn}
   title="Welcome back"
   subtitle="Log in to your account"
   footer={
    <>
     Don't have an account?{" "}
     <Link
      to={"/register" + (returnTo !== "/" ? "?returnTo=" + encodeURIComponent(returnTo) :
""")}
      className="text-primary font-medium hover:underline"
```

```
      >
       Create one
      </Link>
     </>
    }
   >
    <Button
     variant="outline"
     className="w-full h-12 text-sm font-medium mb-6"
     onClick={handleGoogle}
    >
     <GoogleIcon className="w-5 h-5 mr-2" />
     Continue with Google
    </Button>

    <div className="relative mb-6">
     <div className="absolute inset-0 flex items-center">
      <div className="w-full border-t border-border" />
     </div>
     <div className="relative flex justify-center text-xs uppercase">
      <span className="bg-card px-3 text-muted-foreground">or</span>
     </div>
    </div>

    {error && (
     <div className="mb-4 p-3 rounded-lg bg-destructive/10 text-destructive text-sm">
      {error}
     </div>
    )}

    <form onSubmit={handleSubmit} className="space-y-4">
     <div className="space-y-2">
      <Label htmlFor="email">Email</Label>
      <div className="relative">
       <Mail className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-
foreground" aria-hidden="true" />
       <Input
        id="email"
        type="email"
        autoComplete="email"
        autoFocus
        placeholder="you@example.com"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        className="pl-10 h-12"
        required
       />
      </div>
     </div>
     <div className="space-y-2">
      <div className="flex items-center justify-between">
       <Label htmlFor="password">Password</Label>
```

```
        <Link to="/forgot-password" className="text-xs text-primary hover:underline">
          Forgot password?
        </Link>
      </div>
      <div className="relative">
        <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-
foreground" aria-hidden="true" />
        <Input
          id="password"
          type="password"
          autoComplete="current-password"
          placeholder="••••••••"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          className="pl-10 h-12"
          required
        />
      </div>
    </div>
    <Button type="submit" className="w-full h-12 font-medium" disabled={loading}>
      {loading ? (
        <>
          <Loader2 className="w-4 h-4 mr-2 animate-spin" />
          Logging in...
        </>
      ) : (
        "Log in"
      )}
    </Button>
   </form>
  </AuthLayout>
 );
}
```